



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



WRITE-AHEAD LOGGING I RECOVERY MEHANIZMI U POSTGRESQL-U

- SEMINARSKI RAD -

Studijski program: Računarstvo
i informatika

Modul: Bezbednost računarskih
sistema

Predmet: Sistemi za upravljanje
bazama podataka

Student:

Andrea Valčić, br. Ind. 1928

Profesor:

prof. dr Aleksandar Stanimirović

Niš, Jun 2026. godine

SADRŽAJ

1. UVOD	4
2. TEORIJSKE OSNOVE WRITE-AHEAD LOGGING MEHANIZMA	5
2.1. ISTORIJAT I MOTIVACIJA	5
2.2. DEFINICIJA I OSNOVNO PRAVILO	5
2.3. ARHITEKTURA WAL SISTEMA U PostgreSQL-U	5
2.4. PROCES ZAPISIVANJA WAL PODATAKA	6
2.5. LOG SEQUENCE NUMBER KAO IDENTIFIKATOR STANJA	6
2.6. WAL I ACID SVOJSTVA	7
2.7. KONFIGURACIONI PARAMETRI	7
3. REDO I UNDO OPERACIJE U PostgreSQL KONTEKSTU	7
3.1. ULOGA REDO I UNDO U RECOVERY MEHANIZMU	7
3.2. REDO OPERACIJA	8
3.3. UNDO OPERACIJA - PostgreSQL SPECIFIČNOST	9
3.4. POREĐENJE PRISTUPA: EKSPlicitNI UNDO LOG VS MVCC	10
3.5. WAL ZAPISI ZA UNDO SCENARIJE	10
3.6. PRAKTIČNE IMPLIKACIJE	10
4. MULTI-VERSION CONCURRENCY CONTROL	11
4.1. OSNOVNI PRINCIP	11
4.2. IMPLEMENTACIJA MVCC U PostgreSQL-U	11
4.3. VIDLJIVOST TUPLE-A	12
4.4. MVCC I UPDATE OPERACIJA	12
4.5. MVCC I IZOLACIONI NIVOI	13
4.6. DEAD TUPLES I VACUUM	13
4.7. VEZA IZMEĐU MVCC I WAL	14
5. CHECKPOINT MEHANIZAM I UPRAVLJANJE WAL SEGMENTIMA	14
5.1. PROBLEM BEZ CHECKPOINT MEHANIZMA	14
5.2. DEFINICIJA I ULOGA CHECKPOINT-A	14
5.3. PROCES IZVRŠAVANJA CHECKPOINT-A	15
5.4. OKIDAČI CHECKPOINT OPERACIJE	15
5.5. UTICAJ CHECKPOINT-A NA PERFORMANSE	16
5.6. WAL SEGMENTI I ŽIVOTNI CIKLUS	16
5.7. WAL ARCHIVING	17
5.8. OPTIMALNO PODEŠAVANJE CHECKPOINT PARAMETARA	18
6. CRASH RECOVERY I POINT-IN-TIME RECOVERY	18
6.1. SCENARIJI PADA SISTEMA	18
6.2. CRASH RECOVERY PROCES	19
6.3. DIJAGNOSTIKA CRASH RECOVERY-A	20
6.4. POINT-IN-TIME RECOVERY	20
6.5. PREDUSLOVI ZA PITR	20
6.6. PITR RECOVERY PROCES	21
6.7. TIMELINE MEHANIZAM	21
6.8. POREĐENJE CRASH RECOVERY U PITR	22
6.9. PRAKTIČNE PREPORUKE	22
7. IMPLEMENTACIJA PRAKTIČNOG DELA	23
7.1. ARHITEKTURA I TEHNOLOŠKI STACK	23
7.2. MODEL SLOJ I DOMENSKI ENTITETI	23
7.3. MERENJE WAL OPERACIJA - SERVISNI I REPOSITORY SLOJ	25
7.4. ANALIZA REZULTATA I DISKUSIJA	27

7.4.1 REDO - INSERT i WAL trajnost	27
7.4.2 UPDATE i MVCC overhead	28
7.4.3 MVCC Rollback.....	28
7.4.4 WAL generation komparacija.....	29
7.4.5 CASCADE WAL amplifikacija.....	30
7.4.6 MVCC akumulacija dead tuples.....	31
7.4.7 Checkpoint i recovery point.....	32
8. ZAKLJUČAK.....	33
LITERATURA	35

1. UVOD

Savremeni sistemi za upravljanje relacionim bazama podataka suočavaju se sa osnovnim izazovom: kako garantovati konzistentnost i trajnost podataka u uslovima neočekivanih kvarova hardvera, prekida napajanja ili grešaka softvera, a pritom ne žrtvovati performanse sistema. Odgovor na ovaj izazov je u tehnici koja predstavlja jedan od stubova svake ozbiljne relacione baze podataka - Write-Ahead Logging, poznat pod akronimom WAL.

WAL princip, formalizovan u seminalnom radu Mohan i saradnika "ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging" (1992), postavlja jednostavno ali moćno pravilo: svaka izmena podataka mora biti najpre zapisana u log pre nego što se primeni na stvarne podatke na disku. Ovaj jednostavan princip ima značajne implikacije za arhitekturu sistema, performanse i mogućnosti oporavka.

PostgreSQL, kao jedan od najstarijih i najrasprostranjenijih open-source sistema za upravljanje bazama podataka, implementira WAL mehanizam koji se smatra industrijskim standardom po robustnosti i bogatstvu funkcionalnosti. Za razliku od nekih komercijalnih sistema koji koriste eksplicitni UNDO log za rukovanje rollback operacijama, PostgreSQL se oslanja na MVCC (Multi-Version Concurrency Control) arhitekturu, koja donosi specifičan pristup upravljanju konkurentnim transakcijama i oporavku od grešaka.

Cilj ovog seminarskog rada je da teorijski objasni i praktično demonstrira WAL mehanizme u PostgreSQL sistemu, sa fokusom na sledeće aspekte:

- Razumevanje WAL principa i njegovu ulogu u garantovanju ACID svojstava transakcija
- Analizu REDO operacija kao mehanizma oporavka od pada sistema
- Razumevanje uloge MVCC arhitekture kao zamene za eksplicitni UNDO log
- Merenje i analizu količine WAL podataka koje generišu različite vrste operacija
- Razumevanje checkpoint mehanizma kao ključne komponente strategije oporavka
- Demonstraciju crash recovery scenarija i Point-in-Time Recovery mogućnosti

U praktičnom delu rada demonstrirani su WAL mehanizmi na stvarnim podacima kroz merljive eksperimente. Količina WAL podataka generisana operacijama unosa, izmene i brisanja analizirana je i upoređena radi utvrđivanja njihovog uticaja na opterećenje sistema. Efekti rollback operacija prikazani su kroz analizu neuspešnih transakcija koje ostavljaju fizičke tragove nevidljive drugim korisnicima, a čije je nakupljanje moguće pratiti putem sistemskih kataloga. Uloga checkpoint mehanizma predstavljena je kroz njegov uticaj na određivanje tačke oporavka i trajanje restauracije podataka nakon pada sistema.

Rad je organizovan u osam poglavlja. Nakon uvoda, u poglavljima 2 do 6 predstavljene su teorijske osnove WAL mehanizma, REDO/UNDO operacija, MVCC arhitekture, checkpoint mehanizma i crash recovery scenarija. U poglavlju 7 prikazana je implementacija praktičnog dela zajedno sa rezultatima eksperimenata, dok je poglavlje 8 posvećeno zaključnim razmatranjima i osvrtu na ostvarene rezultate.

2. Teorijske osnove Write-Ahead Logging mehanizma

2.1. Istorijat i motivacija

Potreba za pouzdanim mehanizmom oporavka transakcija javila se zajedno sa prvim sistemima za upravljanje bazama podataka. Rani sistemi koristili su jednostavne tehnike beleženja izmena, ali bez formalne teorijske osnove koja bi garantovala konzistentnost u svim scenarijima kvara. Prekretnicu predstavlja rad C. Mohana i saradnika iz 1992. godine - ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) - koji je postavio formalnu osnovu za WAL mehanizme kakve poznajemo danas. ARIES algoritam uvodi tri ključna principa: pisanje log zapisa pre izmene podataka, ponavljanje istorije pri oporavku i beleženje izmena pri rollback operacijama. Gotovo svi savremeni relacioni sistemi, uključujući PostgreSQL, IBM DB2 i Microsoft SQL Server, zasnivaju svoje mehanizme oporavka na ovim principima.

2.2. Definicija i osnovno pravilo

Write-Ahead Logging je tehnika upravljanja transakcijama koja se zasniva na jednom fundamentalnom pravilu::

Svaki log zapis koji opisuje izmenu podataka mora biti trajno zapisan na disk pre nego što se sama izmena primeni na podatkovne stranice.

Ovo pravilo, poznato kao WAL pravilo ili write-ahead rule, ima dve direktne posledice. Prvo, sistem nikada ne može da izgubi potvrđenu transakciju jer je njen trag u log-u uvek ispred stvarnih izmena podataka. Drugo, pri oporavku od pada sistema uvek postoji dovoljan broj informacija u log-u da se rekonstruiše konzistentno stanje baze podataka.

Naziv "write-ahead" direktno opisuje redosled operacija: log se piše unapred, pre podataka.

2.3. Arhitektura WAL sistema u PostgreSQL-u

PostgreSQL implementira WAL kao sekvencijalni log fajl koji se čuva u direktorijumu pg_wal. Svaki zapis u ovom log-u identifikovan je jedinstvenom vrednošću koja se naziva LSN - Log Sequence Number. LSN je monotono rastuća vrednost formata X/YYYYYYYYY koja jednoznačno identifikuje poziciju u WAL toku.

WAL tok:

```
[LSN 0/3A1B000] INSERT u dvi_report_suite
[LSN 0/3A1C840] UPDATE status = 'COMPLETED'
[LSN 0/3A2F100] COMMIT
[LSN 0/3A3E200] CHECKPOINT
```

WAL u PostgreSQL-u organizovan je u segmente veličine 16 MB po podrazumevanoj konfiguraciji. Svaki segment nosi ime koje kodira vremensku liniju i LSN poziciju, što omogućava jednostavnu navigaciju kroz istoriju izmena.

Centralni konfiguracioni parametar koji određuje nivo detalja WAL zapisa je `wal_level`, koji može imati vrednosti:

- `minimal` - Minimalne informacije, ne podržava replikaciju ni PITR
- `replica` - Podržava streaming replikaciju i Point-in-Time Recovery
- `logical` - Logička replikacija, najveći obim log podataka

2.4. Proces zapisivanja WAL podataka

Kada transakcija izvrši izmenu podataka, PostgreSQL ne piše odmah na disk. Umesto toga, izmena prolazi kroz sledeće faze:

1. `WAL buffer` - Izmena se najpre zapisuje u WAL bafer u memoriji (`wal_buffers`, podrazumevano 16 MB). Ovo je brza operacija koja ne uključuje disk I/O.
2. `Flush na disk` - U trenutku potvrde transakcije (`COMMIT`), WAL bafer se obavezno ispisuje na disk. Ovaj trenutak garantuje trajnost transakcije. Parametar `synchronous_commit` kontroliše striktnost ovog koraka.
3. `Izmena shared_buffers` - Stvarne podatkovne stranice u `shared_buffers` memoriji se modifikuju i označavaju kao `dirty` (prljave).
4. `Bgwriter/Checkpoint` - Dirty stranice se periodično ispisuju na disk kroz background writer proces ili na checkpoint-u.

Ključno zapažanje je da u koraku 2 podaci ne moraju biti na disku, dovoljno je da WAL bude na disku. Ovo je suština WAL principa.

2.5. Log Sequence Number kao identifikator stanja

LSN vrednost ima centralnu ulogu u WAL mehanizmu jer jednoznačno identifikuje stanje baze podataka u svakom trenutku. PostgreSQL izlaže nekoliko funkcija za rad sa LSN vrednostima:

```
-- Trenutna pozicija WAL insert pointera
SELECT pg_current_wal_insert_lsn();
-- Rezultat: 0/3A1C840

-- Razlika između dve LSN pozicije (u bajtovima)
SELECT pg_wal_lsn_diff('0/3A1C840', '0/3A1B000');
-- Rezultat: 6208

-- Naziv WAL segmenta za datu LSN poziciju
SELECT pg_walfile_name('0/3A1C840');
-- Rezultat: 000000010000000000000003
```

Slika 1. Prikaz funkcija za rad sa LSN vrednostima

Merenje razlike između LSN vrednosti pre i posle izvršavanja operacije daje precizan uvid u količinu WAL podataka koju je ta operacija generisala, što je osnova eksperimenata u praktičnom delu ovog rada.

2.6. WAL i ACID svojstva

WAL mehanizam direktno podupire dva od četiri ACID svojstva transakcija:

- Atomičnost (Atomicity) - Ako sistem padne usred izvršavanja transakcije, WAL sadrži dovoljno informacija da se utvrdi da transakcija nije bila potvrđena. Sve njene delimične izmene se ignorišu pri oporavku.
- Trajnost (Durability) - Kada klijent dobije potvrdu o uspešnom izvršavanju transakcije (COMMIT), WAL zapis je već na disku. Čak i u slučaju pada sistema neposredno nakon potvrde, transakcija će biti obnovljena pri sledećem pokretanju.

Konzistentnost i izolacija ostvaruju se kroz MVCC mehanizam koji je opisan u poglavlju 4.

2.7. Konfiguracioni parametri

Performanse i pouzdanost WAL mehanizma u PostgreSQL-u kontrolišu se kroz skup parametara u postgresql.conf sa njihovim preporučenim vrednostima navedenim u zagradama:

- wal_level - Nivo detalja WAL zapisa (replica)
- synchronous_commit - Sinhronizacija sa diskom (on)
- wal_buffers - Veličina WAL bafera u memoriji (16MB - 64MB)
- max_wal_size - Maksimalna veličina WAL pre checkpoint-a (1GB - 4GB)
- min_wal_size - Minimalna rezervisana veličina WAL 80MB
- checkpoint_timeout - Maksimalni interval između checkpoint-a (5 min - 15min)

Podešavanje ovih parametara predstavlja balans između performansi (manji synchronous_commit, veći wal_buffers) i sigurnosti podataka (striktniji synchronous_commit, češći checkpoint-i).

3. REDO i UNDO operacije u PostgreSQL kontekstu

3.1. Uloga REDO i UNDO u recovery mehanizmu

Svaki sistem za upravljanje bazama podataka koji implementira WAL mora rešiti dva osnovna problema pri oporavku od pada: Šta uraditi sa transakcijama koje jesu bile potvrđene, ali čije izmene možda nisu bile upisane na disk? Šta uraditi sa transakcijama koje nisu bile potvrđene u trenutku pada?

Odgovor na prvi problem je REDO operacija, ponavljanje potvrđenih izmena iz WAL-a. Odgovor na drugi problem je UNDO operacija, poništavanje izmena koje nisu potvrđene. Ova dva mehanizma zajedno garantuju konzistentno stanje baze podataka nakon bilo kog scenarija kvara.

3.2. REDO operacija

REDO operacija predstavlja ponavljanje istorije gde PostgreSQL čita WAL zapise od poslednjeg checkpoint-a i primenjuje sve pronađene izmene na podatkovne stranice, bez obzira na to da li su te transakcije bile potvrđene ili ne. Tek nakon što je istorija u potpunosti ponovljena, sistem pristupa poništavanju nepotvrđenih transakcija.

Ovaj pristup, direktno preuzet iz ARIES algoritma, na prvi pogled može izgledati neefikasno zašto što ponavlja i nepotvrđene transakcije. Razlog toga je arhitekturne prirode: pri čitanju WAL-a sistem ne zna unapred koji zapisi pripadaju potvrđenim, a koji nepotvrđenim transakcijama. Jednostavnije je da se ponovi sve transakcije, a zatim poništiti nepotvrđene

Proces REDO oporavka izvršava se u 6 koraka:

1. Pronađi poslednji checkpoint u pg_control fajlu
2. Čitaj WAL od checkpoint.redo_lsn 3
3. Za svaki WAL zapis:
 1. Primeni izmenu na podatkovnu stranicu (bez obzira na status transakcije)
4. Nastavi do kraja WAL-a
5. Sve transakcije bez COMMIT zapisa → UNDO faza
6. Baza je konzistentna i dostupna

U praksi, PostgreSQL kontinuirano piše WAL zapise koji opisuju svaku izmenu podataka. Svaki zapis sadrži dovoljno informacija da se izmena može idempotentno ponoviti, primena istog REDO zapisa više puta daje isti rezultat. Ova osobina je ključna za robustnost recovery mehanizma.

Količina WAL podataka koji nastaju pri različitim operacijama direktno određuje trajanje REDO faze oporavka. Praktični deo ovog rada meri upravo ovu veličinu - svaki INSERT u tabelu finansijskih izveštaja generiše konkretan, merljiv WAL zapis koji bi bio replay-ovan u scenariju oporavka.

3.3. UNDO operacija - PostgreSQL specifičnost

Za razliku od sistema kao što su Oracle ili Microsoft SQL Server koji održavaju eksplicitni UNDO log (rollback segment) za poništavanje nepotvrđenih izmena, PostgreSQL primenjuje u osnovi drugačiji pristup zasnovan na MVCC arhitekturi.

U klasičnom UNDO pristupu, sistem pri rollback-u čita UNDO log i fizički vraća podatke na prethodno stanje. PostgreSQL ovaj problem rešava elegantnije: nepotvrđene izmene nikada nisu ni bile vidljive ostalim transakcijama, pa ih nije ni potrebno fizički poništavati.

PostgreSQL nema eksplicitni UNDO log. MVCC mehanizam PostgreSQL-a dodeljuje svakom zapisu (tuple-u) dva atributa:

- xmin - identifikator transakcije koja je kreirala tuple
- xmax - identifikator transakcije koja je obrisala ili izmenila tuple

Kada transakcija pokuša da pročita podatke, PostgreSQL proverava da li je transakcija identifikovana sa xmin bila potvrđena u trenutku čitanja. Ako nije - tuple je nevidljiv. Na ovaj način, rollback se svodi na jednostavno označavanje transakcije kao prekinute u commit log-u (pg_xact), bez ikakve modifikacije podatkovnih stranica.

Scenario rollback-a:

BEFORE rollback:

tuple: xmin=1234, xmax=NULL, data='novi_zapis'
pg_xact: xid=1234 → IN_PROGRESS

AFTER rollback:

tuple: xmin=1234, xmax=NULL, data='novi_zapis' ← fizički nepromenjen
pg_xact: xid=1234 → ABORTED

Rezultat: tuple postoji ali je NEVIDLJIV svim transakcijama

Posledica ovog pristupa je nakupljanje dead tuples - fizički prisutnih, ali transakcijski nevidljivih zapisa. Ovo je cena koju PostgreSQL plaća za eliminaciju eksplicitnog UNDO loga. Dead tuples nastaju u tri situacije:

1. Rollback - tuple je upisan ali transakcija nije potvrđena
2. UPDATE - MVCC čuva staru verziju tuple-a kao dead tuple
3. DELETE - tuple je označen kao obrisani, ali fizički ostaje

Proces VACUUM periodično čisti dead tuples i oslobađa prostor. Autovacuum daemon automatski pokreće ovaj proces kada procenat dead tuples pređe konfigurisani prag (podrazumevano 20%).

3.4. Poređenje pristupa: eksplicitni UNDO log vs MVCC

Na slici 2 prikazana je tabela u kojoj sa poređenjem karakteristika između eksplicitnog UNDO log i PostgreSQL MVCC pristupa.

Karakteristika	Eksplicitni UNDO log	PostgreSQL MVCC
Rollback mehanizam	Fizičko vraćanje podataka	Označavanje xid kao ABORTED
Vidljivost necommit. izmena	Privremeno vidljive	Nikad vidljive
Troškovi rollback-a	Proporcionalni veličini izmene	Konstantni (O(1))
Naknadni troškovi	Manji	Dead tuples → VACUUM potreban
Kompleksnost implementacije	Visoka	Niža

Slika 2. Poređenje karakteristika eksplicitnog UNDO log i PostgreSQL MVCC pristupa

Ključna prednost PostgreSQL pristupa je da je rollback operacija praktično besplatan, ne zahteva nikakvo pisanje podataka, samo upis jednog bita u commit log. Cena se plaća kasnije kroz VACUUM operacije.

3.5. WAL zapisi za UNDO scenarije

Iako PostgreSQL ne koristi eksplicitni UNDO log, WAL ipak sadrži abort zapise za prekinute transakcije. Ovi zapisi su neophodni pri crash recovery-u. Ako sistem padne dok je transakcija bila u toku, pri oporavku se njen xid označava kao ABORTED na osnovu odsustva COMMIT zapisa u WAL-u..

```
-- pg_waldump ispis za rollback scenario:
rmgr: Transaction len (rec/tot):    46/    46, tx:    1234
desc: ABORT 2026-06-09 21:47:54.462
```

Slika 3. pg_waldump ispis ABORT zapisa za prekinutu transakciju (xid: 1234)

Odsustvo COMMIT zapisa za datu transakciju ekvivalentno je eksplicitnom ABORT zapisu jer PostgreSQL tretira oba slučaja identično pri recovery procesu.

3.6. Praktične implikacije

Razumevanje razlike između REDO i UNDO mehanizama ima direktne praktične implikacije.

Za performanse, UPDATE operacija u PostgreSQL-u generiše značajno više WAL podataka od ekvivalentne operacije u sistemima sa UNDO logom, jer MVCC zahteva čuvanje cele nove verzije tuple-a. Ovo se može izmeriti i biće prikazano u praktičnom delu rada.

Za kapacitet, Dead tuples nakupljeni od UPDATE i DELETE operacija povećavaju fizičku veličinu tabela i zahtevaju redovno VACUUM čišćenje. Praćenje broja dead tuples kroz pg_stat_user_tables daje uvid u intenzitet WAL aktivnosti na tabeli.

Za recovery vreme, trajanje REDO faze oporavka proporcionalno je količini WAL-a koji je nastao od poslednjeg checkpoint-a. Optimalno podešavanje checkpoint parametara direktno utiče na maksimalno vreme oporavka sistema.

4. Multi-Version Concurrency Control

4.1. Osnovni princip

Konkurentni pristup podacima predstavlja jedan od centralnih izazova u sistemima za upravljanje bazama podataka. Tradicionalni pristup zasnovan na zaključavanju (locking) rešava problem konkurentnosti na logički jasan način. Dok jedna transakcija čita ili menja podatke, ostale čekaju. Međutim, ovaj pristup uvodi značajne performansne probleme u okruženjima sa visokim brojem konkurentnih korisnika.

PostgreSQL čita sve redove iz jedne tabele, koja se naziva build input ili unutrašnja tab PostgreSQL rešava ovaj problem kroz MVCC (Multi-Version Concurrency Control) mehanizam koji čuva višestruke verzije istog zapisa i omogućava čitaocima da ne blokiraju pisce, i obrnuto. Osnovno pravilo MVCC-a glasi:

Čitaoci ne blokiraju pisce. Pisci ne blokiraju čitaoce.

Ovaj princip ima direktnu vezu sa WAL mehanizmom. Svaka nova verzija tuple-a generiše WAL zapis, što znači da MVCC arhitektura direktno određuje obim WAL podataka koji nastaju pri operacijama izmene.

4.2. Implementacija MVCC u PostgreSQL-u

PostgreSQL implementira MVCC kroz sistem sistemskih atributa koji se čuvaju uz svaki tuple u tabeli. Svaki zapis ima:

- xmin - identifikator transakcije (XID) koja je kreirala ovaj tuple
- xmax - identifikator transakcije koja je obrisala ili zamenila ovaj tuple (NULL ako je aktivan)
- ctid - fizička lokacija tuple-a u tabeli (blok i offset)
- cmin/cmax - redni broj komande unutar transakcije

```
SELECT xmin, xmax, ctid, id, status
FROM dvi_report_suite
LIMIT 3;
```

xmin	xmax	ctid	id	status
1821	0	(0,1)	abc123def456789012345	ACTIVE
1821	0	(0,2)	def456abc789012345678	ACTIVE
1822	1823	(0,3)	ghi789jkl012345678901	COMPLETED

Slika 4. Pregled sistemskih atributa

U trećem redu, xmax=1823 ukazuje da je transakcija 1823 izmenila ili obrisala ovaj tuple. Stara verzija je sačuvana kao dead tuple dok nova verzija postoji na drugoj fizičkoj lokaciji.

4.3. Vidljivost tuple-a

Kada transakcija čita podatke, PostgreSQL kreira snapshot, tj. snimak stanja baze koji definiše koje su transakcije bile aktivne, potvrđene ili prekinute u trenutku početka čitanja. Na osnovu ovog snimka, za svaki tuple se proverava vidljivost:

Tuple je VIDLJIV ako:

1. xmin je potvrđen (COMMITTED) pre trenutnog snapshot-a
2. xmax je NULL (tuple nije izmenjen/obrisan)
ILI
xmax nije potvrđen u trenutnom snapshot-u
(izmena/brisanje se desilo posle snapshot-a)

Ova logika garantuje da svaka transakcija vidi konzistentnu sliku podataka iz trenutka svog početka, bez obzira na to šta se dešava u konkurentnim transakcijama.

4.4. MVCC i UPDATE operacija

Najvažnija implikacija MVCC arhitekture za WAL mehanizam je ponašanje UPDATE operacije. Za razliku od sistema sa in-place update (direktna izmena podataka na istom mestu), PostgreSQL UPDATE je zapravo kombinacija logičkog DELETE i INSERT:

```
UPDATE dvi_report_suite SET status = 'COMPLETED' WHERE id = 'abc123...';
```

Slika 5. Primer UPDATE metode

Fizički se dešava:

1. Stari tuple: xmax = current_xid (označen kao obrisani)
2. Novi tuple: INSERT sa novim vrednostima, xmin = current_xid
3. WAL zapisuje: staru stranicu (sa xmax) + novu stranicu (novi tuple)

Upravo zato UPDATE generiše značajno više WAL podataka od INSERT jer sistem mora zapisati informacije o obe verzije tuple-a. Ovo je osnovna karakteristika MVCC arhitekture koja ima direktne implikacije za kapacitet WAL-a i performanse sistema.

PostgreSQL uvodi optimizaciju poznatu kao HOT update (Heap-Only Tuple) koja smanjuje WAL overhead UPDATE operacija. HOT update je moguć kada su ispunjena dva uslova:

1. UPDATE ne menja nijednu indeksiranu kolonu
2. Nova verzija tuple-a može stati na istu stranicu kao stara verzija

U ovom slučaju indeksi se ne menjaju, pa nije potrebno njihovo ažuriranje. Zbog toga se u WAL beleže samo promene nad podacima, dok regularni UPDATE generiše WAL zapise i za podatke i za indekse. Rezultat je manja količina WAL podataka i efikasnije izvršavanje izmena.

Praćenje HOT update efikasnosti moguće je kroz `n_tup_hot_upd` kolonu u `pg_stat_user_tables` sistemskom katalogu.

4.5. MVCC i izolacioni nivoi

PostgreSQL implementira tri izolaciona nivoa kroz MVCC mehanizam. Na nivou `READ COMMITTED`, snapshot se kreira na početku svake komande, što znači da ista transakcija može videti različite verzije podataka tokom svog izvršavanja. Ovaj nivo je podložan non-repeatable read fenomenu. Na nivou `REPEATABLE READ`, snapshot se kreira jednom na početku transakcije i ostaje nepromenjen do njenog završetka, čime se eliminiše non-repeatable read, ali sistem i dalje nije zaštićen od phantom read pojave. Najviši nivo, `SERIALIZABLE`, koristi isti pristup kreiranja snapshota kao `REPEATABLE READ`, ali dodatno primenjuje SSI (Serializable Snapshot Isolation) mehanizam koji detektuje i sprečava sve anomalije, garantujući punu serializabilnost izvršavanja konkurentnih transakcija.

Važno je napomenuti da PostgreSQL ne implementira `READ UNCOMMITTED` nivo. Minimalni nivo je `READ COMMITTED`, što znači da necommittovane izmene nikada nisu vidljive, što je direktna posledica MVCC dizajna.

4.6. Dead tuples i VACUUM

Svaka `UPDATE` i `DELETE` operacija, kao i svaki rollback, ostavlja takozvani dead tuple u tabeli. Akumulacija ovih dead tuples tokom vremena ima nekoliko negativnih efekata na performanse sistema. Prvo, dolazi do povećanja fizičke veličine tabele, jer dead tuples zauzimaju prostor koji ne može biti iskorišćen za nove zapise. Drugo, sekvencijalno skeniranje postaje sporije, budući da PostgreSQL mora da preskače dead tuples prilikom čitanja podataka. Treće, dolazi do bloatanja indeksa, s obzirom na to da indeksi i dalje sadrže pokazivače na dead tuples koji više nisu aktivni, što nepotrebno povećava njihovu veličinu i usporava pretrage.

`VACUUM` operacija rešava ovaj problem kroz dva koraka: označava dead tuples kao slobodan prostor (regular `VACUUM`) ili fizički kompaktira tabelu vraćajući prostor operativnom sistemu (`VACUUM FULL`).

```
SELECT relname, n_live_tup, n_dead_tup,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS
FROM pg_stat_user_tables
WHERE relname = 'dvi_report_suite';
```

relname	n_live_tup	n_dead_tup	dead_pct
dvi_report_suite	50000	847	1.66

Slika 6. Stanje dead duples u tabeli

Autovacuum daemon automatski pokreće VACUUM kada dead_pct pređe konfigurisan prag, što je obično 20% za standardne tabele.

4.7. Veza između MVCC i WAL

MVCC i WAL su neraskidivo povezani mehanizmi u PostgreSQL-u. Svaka nova verzija tuple-a koja nastaje kao posledica MVCC semantike UPDATE operacije mora biti zapisana u WAL, što stvara direktnu vezu između intenziteta izmena podataka i ukupnog obima WAL podataka. Visok UPDATE intenzitet dovodi do nastanka većeg broja verzija tuple-a, što za posledicu ima porast broja WAL zapisa i ubrzani rast WAL fajlova. Kako bi sistem ostao stabilan u takvim uslovima, javlja se potreba za češćim izvršavanjem checkpoint operacija. Istovremeno, svaka UPDATE operacija ostavlja dead tuple u tabeli, pa veći intenzitet izmena direktno uslovljava i potrebu za češćim pokretanjem VACUUM operacija radi održavanja prostora i performansi.

Razumevanje ove niza efekata ključno je za pravilno dimenzionovanje i podešavanje PostgreSQL sistema u produkcijskom okruženju, što će biti ilustrovano kroz merenja u praktičnom delu rada.

5. Checkpoint mehanizam i upravljanje WAL segmentima

5.1. Problem bez checkpoint mehanizma

WAL mehanizam garantuje trajnost podataka upisivanjem svih izmena u sekvencijalni log pre primene na podatkovne stranice. Međutim, ovo stvara potencijalni problem pri oporavku: ako se nikada ne uspostavi veza između log zapisa i stvarnog stanja podatkovnih stranica na disku, sistem bi pri svakom oporavku morao čitati WAL od samog početka što bi za baze u produkcijskom radu moglo značiti čitanje terabajta log podataka.

Checkpoint mehanizam rešava ovaj problem definisanjem periodičnih tačaka u kojima je garantovano da su sve izmene iz WAL-a upisane na disk. Pri oporavku, sistem ne mora čitati WAL od početka. Dovoljno je početi od poslednjeg checkpoint-a.

5.2. Definicija i uloga checkpoint-a

Checkpoint je operacija koja obezbeđuje da sve dirty stranice (stranice u shared_buffers memoriji koje su izmenjene ali još nisu upisane na disk) budu trajno zapisane na disk. Nakon završetka checkpoint-a primenjuje se sledeće pravilo:

Sve izmene koje su se desile pre checkpoint LSN pozicije su trajno na disku. Pri oporavku od pada, WAL se čita samo od redo_lsn checkpoint-a.

Checkpoint se čuva u fajlu `pg_global/pg_control` koji PostgreSQL čita pri svakom pokretanju. Ovaj fajl sadrži:

```
SELECT checkpoint_lsn, redo_lsn, timeline_id,
       checkpoint_time, next_xid
FROM pg_control_checkpoint();
```

checkpoint_lsn	redo_lsn	timeline_id	checkpoint_time
0/3A5F000	0/3A5F000	1	2026-06-09 22:38:00.000+02

Slika 7. Sadržaj `pg_global/pg_control`

`redo_lsn` je ključna vrednost. Ona označava tačku od koje PostgreSQL počinje čitanje WAL-a pri crash recovery procesu.

5.3. Proces izvršavanja checkpoint-a

Checkpoint operacija koja se izvršava u jednom trenutku. Da bi se minimizovao uticaj na performanse sistema, PostgreSQL raspoređuje pisanje dirty stranica tokom određenog vremenskog intervala. Proces se odvija u sledećim koracima:

1. Identifikacija dirty stranica - PostgreSQL skenira `shared_buffers` i identifikuje sve stranice koje su izmenjene od poslednjeg checkpoint-a.
2. Postepeno pisanje - Dirty stranice se postupno ispisuju na disk tokom intervala definisanog parametrom `checkpoint_completion_target`. Vrednost 0.9 znači da se pisanje raspoređuje na 90% intervala između dva checkpoint-a, čime se izbegavaju I/O skokovi.
3. WAL zapis checkpoint-a - Kada su sve dirty stranice upisane, u WAL se upisuje checkpoint zapis koji sadrži `redo_lsn` - LSN poziciju od koje počinje recovery.
4. Ažuriranje `pg_control` - Fajl `pg_global/pg_control` se ažurira novim checkpoint LSN vrednostima.

5.4. Okidači checkpoint operacije

Checkpoint se može pokrenuti na nekoliko načina:

- `checkpoint_timeout` - Automatski, po isteku vremenskog intervala (podrazumevano 5 min)
- `max_wal_size` - Automatski, kada WAL dostigne konfigurisan maksimum (podrazumevano 1 GB)
- CHECKPOINT SQL komanda - Ručno pokretanje od strane administratora
- Shutdown - Pri normalnom zaustavljanju servera
- `pg_start_backup()` - Pri inicijalnom backup-u za PITR

Checkpoint pokrenut zbog `max_wal_size` prekoračenja smatra se prinudnim checkpoint-om i može ukazivati na nedovoljno konfigurisanu vrednost ovog parametra za dati workload.

5.5. Uticaj checkpoint-a na performanse

Checkpoint operacija ima dvostruki uticaj na performanse sistema, I/O uticaj i povećanje WAL podataka.

I/O uticaj, tj. pisanje velikog broja dirty stranica istovremeno može izazvati značajan spike u disk write operacijama. Parametar `checkpoint_completion_target` ublažava ovaj efekat raspoređivanjem pisanja.

Prva izmena svake stranice sa podacima nakon checkpoint-a generiše Full Page Image (FPI) i kompletan snimak stranice koji se upisuje u WAL. Razlog je sigurnosne prirode: ako sistem padne usred pisanja stranice na disk (torn page), recovery mora imati kompletan sadržaj stranice, a ne samo delta izmenu.

```
SELECT wal_records, wal_fpi, wal_bytes,
       ROUND(100.0 * wal_fpi / NULLIF(wal_records, 0), 2) AS fpi_pct
FROM pg_stat_wal;
```

wal_records	wal_fpi	wal_bytes	fpi_pct
847293	23841	94823648	2.81

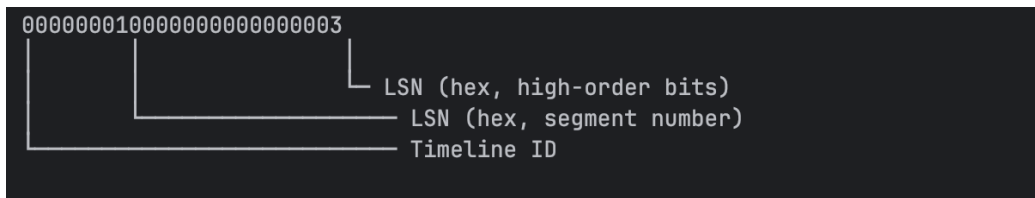
Slika 8. FPI statistika

Visok procenat FPI zapisa može ukazivati na prečeste checkpoint operacije.

5.6. WAL segmenti i životni ciklus

WAL log je organizovan u segmente, tj. u fajlove fiksne veličine koji se čuvaju u direktorijumu `pg_wal`. Podrazumevana veličina segmenta je 16 MB i može se podesiti samo pri inicijalizaciji klastera (`initdb --wal-segsize`).

WAL segment se imenuje po šablonu koji kodira vremensku liniju i LSN poziciju:



Slika 9. Imenovanje WAL segmenata

Životni ciklus WAL segmenta odvija se u 4 faze počevši od prve koja je kreiranje, zatim popunjavanje, arhiviranje i na kraju recikliranje, tj. brisanje.

PostgreSQL ne briše odmah stare WAL segmente. Umesto toga ih reciklira preimenovanjem za buduću upotrebu. Broj segmenata koji se čuvaju reguliše se parametrima `min_wal_size` i `max_wal_size`.

```
SELECT name,
       size,
       modification,
       pg_size_pretty(size) AS size_pretty
FROM pg_ls_waldir()
ORDER BY modification DESC
LIMIT 5;
```

name	size	modification	size_pretty
00000001000000000000000042	16777216	2026-06-09 22:15:33+02:00	16 MB
00000001000000000000000041	16777216	2026-06-09 22:14:12+02:00	16 MB
00000001000000000000000040	16777216	2026-06-09 22:12:58+02:00	16 MB

Slika 10. Pregled WAL segmenata

5.7. WAL archiving

Za potrebe Point-in-Time Recovery (opisanog u poglavlju 6), PostgreSQL podržava WAL archiving, automatsko kopiranje završenih WAL segmenata na sigurnu lokaciju pre njihovog recikliranja.

Archiving se aktivira kroz parametre:

- `wal_level = replica`
- `archive_mode = on`
- `archive_command = 'cp %p /mnt/wal_archive/%f'`

Gde `%p` predstavlja putanju izvornog segmenta, a `%f` ime fajla. Status archiving-a dostupan je kroz sistemski katalog:

```
SELECT archived_count, last_archived_wal,
       last_archived_time, failed_count
FROM pg_stat_archiver;
```

archived_count	last_archived_wal	failed_count
1847	00000001000000000000000041	0

Slika 11. Prikaz sistemskog kataloga

5.8. Optimalno podešavanje checkpoint parametara

Balans između recovery vremena i performansi sistema postiže se podešavanjem checkpoint parametara. Parametar `checkpoint_timeout` definiše vremenski interval između dva checkpoint-a gde manja vrednost znači kraće recovery vreme u slučaju pada sistema, ali i veće I/O opterećenje tokom normalnog rada, dok veća vrednost smanjuje I/O pritisak na račun potencijalno dužeg recovery vremena. Parametar `max_wal_size` kontroliše maksimalnu količinu WAL podataka između dva checkpoint-a. Manja vrednost `max_wal_size` parametra izaziva češće checkpoint-e, dok veća vrednost dopušta ređe checkpoint-e uz cenu većeg prostora koji WAL fajlovi zauzimaju na disku. Parametar `checkpoint_completion_target` određuje koliko vremena PostgreSQL ima na raspolaganju da rasporedi I/O operacije tokom pisanja checkpoint-a. Veća vrednost ovog parametra znači ravnomerniji I/O kroz duži vremenski period, čime se smanjuju trenutni skokovi opterećenja diska.

Za produkcijske sisteme sa visokim write workload-om preporučuje se::

- `checkpoint_timeout = 10min`
- `max_wal_size = 2GB`
- `checkpoint_completion_target = 0.9`

Ova konfiguracija obezbeđuje maksimalno vreme oporavka od oko 10 minuta uz minimalan uticaj checkpoint I/O operacija na normalan rad sistema.

6. Crash recovery i Point-in-Time Recovery

6.1. Scenariji pada sistema

Pad sistema može nastupiti iz različitih razloga: nestanak struje, hardverski kvar, grška operativnog sistema ili prinudno zaustavljanje PostgreSQL procesa. Bez obzira na uzrok, PostgreSQL mora biti u stanju da se oporavi do konzistentnog stanja bez gubitka potvrđenih transakcija.

PostgreSQL razlikuje dva osnovna načina zaustavljanja servera koji imaju različite posledice po integritet podataka. Komanda `pg_ctl stop -m fast` predstavlja čisto zaustavljanje u kome se pre gašenja izvršava finalni checkpoint, čime se svi izmenjeni podaci upisuju na disk i pri sledećem pokretanju recovery nije potreban. Nasuprot tome, komanda `pg_ctl stop -m immediate` brutalno prekida rad servera bez izvršavanja checkpoint-a, što znači da pri sledećem pokretanju PostgreSQL mora proći kroz crash recovery proceduru kako bi doveo bazu u konzistentno stanje. Nestanak struje ili nasilno gašenje procesa putem `kill -9` signala tretiraju se identično kao `immediate` zaustavljanje, sistem nema priliku da izvrši checkpoint, pa je recovery neizbežan pri ponovnom pokretanju.

Jedino u slučaju čistog zaustavljanja (`fast` ili `smart`) PostgreSQL ne mora izvršavati recovery pri sledećem pokretanju. U svim ostalim slučajevima, WAL replay je obavezan.

6.2. Crash recovery proces

Kada PostgreSQL detektuje da prethodno zaustavljanje nije bilo uredno (na osnovu pg_control fajla), automatski pokreće crash recovery bez ikakve intervencije administratora. Proces se odvija u tri faze koje direktno odgovaraju ARIES algoritmu:

Faza 1: Analysis (Analiza) PostgreSQL čita pg_control fajl i pronalazi LSN poziciju poslednjeg checkpoint-a (redo_lsn). Ova vrednost definiše početnu tačku za čitanje WAL-a. Sistem takođe utvrđuje koje su transakcije bile aktivne u trenutku checkpoint-a.

Faza 2: REDO (Ponavljanje istorije) Počevši od redo_lsn, PostgreSQL sekvencijalno čita WAL i primenjuje sve pronađene izmene na podatkovne stranice. Bez obzira na to da li su transakcije bile potvrđene ili ne. Ovaj korak garantuje da nijedna potvrđena izmena nije izgubljena.

```
[0/3A5F000] CHECKPOINT (redo_lsn = ovde počinjemo)
[0/3A6B100] BEGIN xid=5001
[0/3A6B200] INSERT u dvi_report_suite (xid=5001)
[0/3A6B400] COMMIT xid=5001      ← potvrđena, mora biti oporavljena
[0/3A6B500] BEGIN xid=5002
[0/3A6B600] UPDATE dvi_report_suite (xid=5002)
--- SISTEM PADE OVDE ---
                                ← xid=5002 nema COMMIT, biće ignorisana
```

Slika 12. Primer WAL sadržaja pri oporavku

Faza 3: UNDO (Poništavanje nepotvrđenih) Nakon što je istorija u potpunosti ponovljena, PostgreSQL prolazi kroz listu transakcija koje nisu imale COMMIT zapis u WAL-u i označava njihove xid vrednosti kao ABORTED u commit log-u (pg_xact). Zahvaljujući MVCC mehanizmu, ovo je jedina akcija potrebna. Fizičke izmene ostaju kao dead tuples i biće očišćene VACUUM operacijom.

```
Ukupno vreme oporavka ≈ vreme čitanja WAL-a od redo_lsn do kraja
                      = f(veličina WAL-a od poslednjeg checkpoint-a)
```

Slika 13. Zavisnost vremena oporavka od veličine WAL-a nakon checkpoint-a

Ovo je direktna motivacija za pravilno podešavanje checkpoint_timeout i max_wal_size parametara jer manji interval između checkpoint-a znači kraće vreme oporavka.

6.3. Dijagnostika crash recovery-a

PostgreSQL ispisuje detaljan log tokom recovery procesa koji omogućava praćenje napretka:

```
LOG: database system was shut down at 2026-06-09 22:15:33 UTC
LOG: entering standby mode
LOG: redo starts at 0/3A5F000
LOG: consistent recovery state reached at 0/3A5F080
LOG: redo in progress, current LSN 0/3A6B400
LOG: redo done at 0/3A6C100 system usage: ...
LOG: last completed transaction was at log time 2026-06-09 22:16:01
LOG: database system is ready to accept connections
```

Slika 14. PostgreSQL log tokom recovery procesa

Razlika između redo starts at i redo done at LSN vrednosti direktno pokazuje koliko WAL-a je moralo biti replayed, što je proporcionalno trajanju recovery procesa.

6.4. Point-in-Time Recovery

Crash recovery oporavlja bazu do trenutka neposredno pre pada, ali postoje scenariji u kojima je potreban oporavak do specifičnog trenutka u prošlosti. Jedan od najčešćih razloga je slučajno izvršena destruktivna operacija poput DROP TABLE ili DELETE bez WHERE klauzule, gde su podaci nepovratno izgubljeni iako je sistem tehnički u konzistentnom stanju. Sličan problem nastaje kada su pogrešni podaci uneti i potvrđeni kroz COMMIT, što znači da crash recovery ne može pomoći jer je transakcija validno završena. Korupcija podataka uzrokovana greškom u aplikaciji takođe može zahtevati vraćanje na ranije stanje, kao i situacije u kojima je jednostavno potrebno ispitati kako je baza izgledala u određenom trenutku u prošlosti.

Za ove scenarije, crash recovery nije dovoljan. PostgreSQL nudi Point-in-Time Recovery (PITR). To je mehanizam koji omogućava oporavak baze do bilo kog trenutka za koji postoji WAL arhiva.

6.5. Preduslovi za PITR

PITR zahteva dva elementa koji moraju biti pripremljeni pre nastanka problema i bez njih PITR nije moguć. To su:

1. Base backup - kompletan snimak fajlova sa podacima u trenutku koji prethodi željenom recovery cilju:

```
pg_basebackup -D /backup/base -Ft -z -P -U andrea
```

Slika 15. prikaz base backup-a

2. Kontinuirana WAL arhiva - svi WAL segmenti od trenutka base backup-a do željenog recovery trenutka moraju biti sačuvani:

```
wal_level = replica
archive_mode = on
archive_command = 'cp %p /mnt/wal_archive/%f'
```

Slika 16. Konfiguracija u postgresql.conf za WAL arhiviranje i PITR

6.6. PITR recovery proces

Proces oporavka do određenog trenutka u vremenu odvija se kroz nekoliko uzastopnih koraka koji zajedno dovode bazu u željeno stanje.

Prvi korak podrazumeva zaustavljanje PostgreSQL servera ukoliko je aktivan, nakon čega se trenutni sadržaj data direktorijuma briše i zamenjuje prethodno kreiranim base backup-om. Ovim se baza vraća na početno stanje od kojeg će recovery krenuti napred kroz WAL.

Drugi korak je konfiguracija recovery parametara direktno u postgresql.conf fajlu, što je pristup koji koriste PostgreSQL 12 i novije verzije. Ključni parametar je `restore_command`, koji definiše komandu kojom PostgreSQL kopira WAL segmente iz arhive tokom replay procesa. Pored toga, potrebno je definisati recovery cilj, koji može biti izražen kao tačno vreme kroz parametar `recovery_target_time`, kao Log Sequence Number kroz `recovery_target_lsn`, ili kao identifikator transakcije kroz `recovery_target_xid`. Parametar `recovery_target_action` određuje šta se dešava kada sistem dostigne definisani cilj. Najčešće korišćena vrednost je `promote`, koja bazu prevodi u normalan operativni režim.

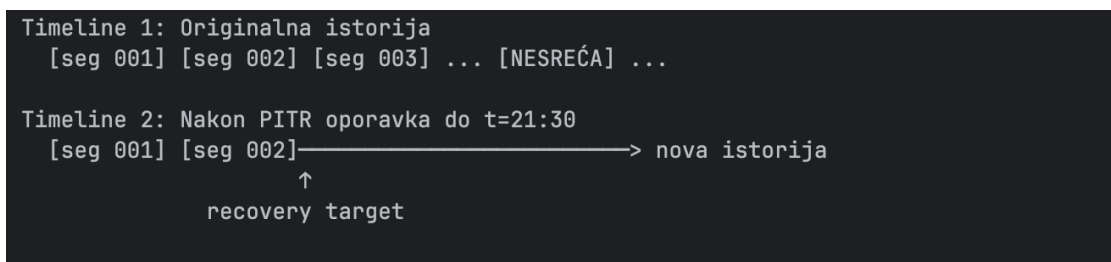
Treći korak je kreiranje `recovery.signal` fajla u data direktorijumu, čije samo prisustvo signalizira PostgreSQL-u da pri sledećem pokretanju treba izvršiti recovery proceduru umesto normalnog starta.

Nakon pokretanja servera u četvrtom koraku, PostgreSQL automatski detektuje `recovery.signal` fajl i počinje da reprodukuje WAL segmente iz arhive redom, sve dok ne dostigne definisani recovery cilj, nakon čega se baza promovise u primarni režim rada.

6.7. Timeline mehanizam

Svaki PITR oporavak kreira novu vremensku liniju, poznatu kao timeline, što je mehanizam koji omogućava PostgreSQL-u da razlikuje originalnu istoriju baze od istorije koja nastaje nakon oporavka. Timeline ID je sastavni deo naziva WAL segmenta i monotono raste sa svakim novim PITR oporavkom, čime svaka recovery operacija dobija jedinstveni identitet u istoriji sistema.

Kada se, na primer, izvrši oporavak do određenog trenutka, originalna istorija ostaje sačuvana kao Timeline 1 i obuhvata sve WAL segmente sve do trenutka pada sistema. Nakon oporavka do definisanog recovery cilja, PostgreSQL kreira Timeline 2 koja deli iste WAL segmente sa originalnom istorijom sve do tačke oporavka, a od tog trenutka nastavlja da gradi novu, nezavisnu istoriju promena.



Slika 17. Nastanak nove timeline nakon PITR oporavka

Ovaj mehanizam ima praktičnu vrednost u situacijama kada je prvobitno izabrani recovery cilj bio pogrešan. Budući da je originalna istorija sačuvana, moguće je pokrenuti novi PITR oporavak i definisati drugačiji cilj, čime nastaje Timeline 3, odnosno oporavak od oporavka. Na taj način PostgreSQL čuva kompletnu istoriju svih recovery operacija i pruža fleksibilnost pri pronalaženju tačnog trenutka u koji je potrebno vratiti bazu.

6.8. Poređenje crash recovery u PITR

Crash recovery i PITR predstavljaju dva fundamentalno različita mehanizma oporavka koji se primenjuju u različitim okolnostima i zahtevaju različite preduslove.

Crash recovery se aktivira automatski kao odgovor na neočekivani pad sistema, kao što su nestanak struje ili nasilno gašenje procesa. Za njegov rad dovoljni su samo WAL segmenti nastali od poslednjeg checkpoint-a, a cilj oporavka je uvek fiksiran (trenutak neposredno pre pada). Konfiguracija je potpuno automatska i ne zahteva intervenciju administratora, a trajanje procesa je tipično kratko, od svega nekoliko sekundi do nekoliko minuta. Važna karakteristika crash recovery-a je da ne dolazi do gubitka podataka za sve transakcije koje su bile potvrđene pre pada sistema.

PITR, sa druge strane, nije namenjen oporavku od tehničkih kvarova već od logičkih grešaka u podacima, poput slučajno izvršenog brisanja ili unosa pogrešnih podataka. Za razliku od crash recovery-a, PITR zahteva postojanje base backup-a i kompletne WAL arhive od trenutka tog backup-a. Prednost ovog mehanizma je fleksibilnost u izboru recovery cilja. Baza se može vratiti u bilo koji trenutak u prošlosti za koji postoje odgovarajući WAL segmenti. Međutim, konfiguracija je ručna i zahteva intervenciju administratora, a trajanje oporavka može biti znatno duže, od nekoliko minuta do nekoliko sati, zavisno od veličine arhive i udaljenosti recovery cilja. Kao neizbežna posledica, sve transakcije potvrđene nakon izabranog recovery cilja se gube.

6.9. Praktične preporuke

Za produkcijsko okruženje, minimalna strategija zaštite podataka podrazumeva kombinaciju dnevnog base backup-a kroz `pg_basebackup`, kontinuiranog WAL archivinga i mesečnog testiranja PITR oporavka. Ovakva kombinacija omogućava postizanje RPO vrednosti bliske nuli, što znači da se u slučaju pada sistem može oporaviti sa minimalnim gubitkom podataka, dok RTO direktno zavisi od trajanja WAL replay procesa.

Odsustvo WAL archivinga drastično narušava ovu zaštitu. U tom slučaju RPO postaje jednak vremenskom intervalu između poslednjeg backup-a i trenutka pada sistema, što u praksi može značiti gubitak podataka od nekoliko sati. Upravo iz tog razloga razumevanje WAL mehanizma nije opciono znanje već fundamentalni uslov za svakog administratora baza podataka koji je odgovoran za integritet i dostupnost produkcijskog sistema.

7. Implementacija praktičnog dela

7.1. Arhitektura i tehnološki stack

Praktični deo rada implementiran je kao REST API aplikacija koja se direktno povezuje na PostgreSQL instancu i izvršava merljive WAL eksperimente na stvarnim domenskim podacima.

Aplikacija je razvijena u programskim jezicima Java 21 i Kotlin 2.3, uz Spring Boot 4.0.6 kao osnovni framework. Za pristup bazi podataka koristi se Spring Data JPA sa Hibernate 7.2 kao ORM slojem, dok je PostgreSQL 16.13. Kao build alat korišćen je Maven sa verzijom 3.9.15.

Arhitektura aplikacije prati standardnu Spring Boot slojevit organizaciju. Controller sloj, smešten u klasi WalController, izlaže REST API krajnje tačke kroz koje se pokreću eksperimenti. Service sloj čine tri klase, WalService koja implementira poslovnu logiku i WAL merenja, WalOperationTracker zadužen za praćenje operacija i WalTestDataFactory koji generiše testne podatke. Repository sloj obezbeđuje pristup bazi kroz tri repozitorijuma: WalRepository, XbrlReportSuiteRepository i AuthUserRepository. JPA entiteti su organizovani u db/model paketu i obuhvataju klase XbrlReportSuite, XbrlAttachment i AuthUser, dok DTO sloj sadrži objekte za prenos podataka kao što su WalOperationResult i RedoOperationResponse

7.2. Model sloj i domenski entiteti

Centralni entitet aplikacije je XbrlReportSuite koji mapira tabelu `dvi_report_suite`. Entitet je posebno interesantan sa WAL aspekta zbog velikog TEXT polja `xbrl` koje čuva XML sadržaj finansijskog izveštaja, što direktno utiče na obim WAL zapisa pri UPDATE operacijama.

```

@Entity
@Table(name = "dvi_report_suite")
@Access(AccessType.PROPERTY)
public class XbrlReportSuite extends BaseEntity {

    private String m_kvnumber; 2 usages
    private String m_name; 2 usages
    private String m_entryPoint; 2 usages
    private String m_xbrl; 2 usages
    private XbrlDocumentStatus m_status = XbrlDocumentStatus.ACTIVE; 4 usages
    private Date m_createdAt = new Date(); 2 usages
    private Date m_modifiedAt; 2 usages
    private Date m_sentAt; 2 usages
    private AuthUser m_createdBy; 2 usages
    private AuthUser m_modifiedBy; 2 usages
    private Date m_completedAt; 2 usages
    private AuthUser m_completedBy; 2 usages
    private AuthUser m_sentBy; 2 usages
    private XbrlDocumentType m_type; 3 usages
    private Integer m_dataYear; 2 usages
    private List<XbrlAttachment> m_attachments = new ArrayList<>(); 2 usages
    private List<XbrlSuiteDelivery> m_deliveryList = new ArrayList<>(); 2 usages

    @OneToMany(mappedBy = "suite", cascade = CascadeType.ALL, fetch = FetchType.LAZY, orphanRemoval = true) 3 usages
    public List<XbrlAttachment> getAttachments() { return m_attachments; }

```

Slika 18. XbrlReportSuite entity klasa

XbrlAttachment čuva binarne podatke (byte[]) koji reprezentuju PDF dokumente:

```

@Entity
@Table(name = "dvi_attachment", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"suiteid", "type"})
})
@Access(AccessType.PROPERTY)
public class XbrlAttachment extends BaseEntity {

    private String m_name; 2 usages
    private XbrlAttachmentType m_type; 2 usages
    private String m_mimeType; 2 usages
    private String m_sha1; 2 usages
    private String m_xbrlCsvSha256; 2 usages
    private byte[] m_file; 2 usages
    private Long m_size; 2 usages
    private XbrlReportSuite m_suite; 2 usages
    private Date m_createdAt; 2 usages
    private AuthUser m_createdBy; 2 usages

    @ManyToOne(fetch = FetchType.LAZY, optional = false, cascade = CascadeType.PERSIST)
    @JoinColumn(
        name = "suiteid",
        nullable = false,
        referencedColumnName = "id",
        foreignKey = @ForeignKey(name = "fk_attachment_suite")
    )
    public XbrlReportSuite getSuite() { return m_suite; }

```

Slika 19. XbrlAttachment entity klasa

Relaciona veza između XbrlReportSuite i XbrlAttachment sa CascadeType.ALL je ključna za demonstraciju CASCADE WAL amplifikacije. Jedan save() poziv generiše WAL zapise za sve povezane entitete.

7.3. Merenje WAL operacija - servisni i repository sloj

Svi direktni PostgreSQL WAL upiti centralizovani su u WalRepository klasi, čime je postignuta čista separacija odgovornosti:

```
@Repository
public class WalRepository {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    public String getCurrentWalInsertLsn() { 6 usages
        return jdbcTemplate.queryForObject(
            sql: "SELECT pg_current_wal_insert_lsn()::text",
            String.class
        );
    }

    public Long calculateWalBytesGenerated(String lsnAfter, String lsnBefore) { 2 usages
        return jdbcTemplate.queryForObject(
            sql: "SELECT pg_wal_lsn_diff(?:pg_lsn, ?::pg_lsn)",
            Long.class,
            lsnAfter,
            lsnBefore
        );
    }

    public Map<String, Object> getTableStats(String tableName) { 3 usages
        return jdbcTemplate.queryForMap(
            sql: "SELECT n.tup_ins, n.tup_upd, n.tup_del, n.tup_hot_upd, " +
                "n.live_tup, n.dead_tup, last_vacuum, last_autovacuum " +
                "FROM pg_stat_user_tables WHERE relname = ?",
            tableName
        );
    }

    public Map<String, Object> getCheckpointInfo() { 1 usage
        return jdbcTemplate.queryForMap(
            sql: "SELECT checkpoint_lsn, redo_lsn, timeline_id, checkpoint_time " +
                "FROM pg_control_checkpoint()"
        );
    }
}
```

Slika 20. WalRepository klasa

WalOperationTracker komponenta automatizuje merenje LSN razlike pre i posle svake operacije:

```

@Component 5 usages
public class WalOperationTracker {

    @Autowired
    private WalRepository walRepository;

    public WalOperationResult track(Runnable operation) { 4 usages
        String lsnBefore = walRepository.getCurrentWalInsertLsn();

        operation.run();

        String lsnAfter = walRepository.getCurrentWalInsertLsn();
        Long bytesGenerated = walRepository.calculateWalBytesGenerated(lsnAfter, lsnBefore);

        return new WalOperationResult(lsnBefore, lsnAfter, bytesGenerated);
    }

    public <T> TrackedResult<T> trackWithResult(java.util.function.Supplier<T> operation) { 4 usages
        String lsnBefore = walRepository.getCurrentWalInsertLsn();

        T result = operation.get();

        String lsnAfter = walRepository.getCurrentWalInsertLsn();
        Long bytesGenerated = walRepository.calculateWalBytesGenerated(lsnAfter, lsnBefore);

        WalOperationResult walResult = new WalOperationResult(lsnBefore, lsnAfter, bytesGenerated);
        return new TrackedResult<>(result, walResult);
    }
}

```

Slika 21. WalOperationTracker klasa

WalService koristi WalOperationTracker za merenje svakog eksperimenta:

```

@Service 2 usages
public class WalService {

    @Autowired
    private XbrlReportSuiteRepository suiteRepository;

    @Autowired
    private AuthUserRepository userRepository;

    @Autowired
    private XbrlAttachmentRepository attachmentRepository;

    @Autowired
    private WalRepository walRepository;

    @Autowired
    private WalOperationTracker tracker;

    @Autowired
    private WalTestDataFactory testDataFactory;

    @Transactional 1 usage
    public RedoOperationResponse demonstrateRedoWithRealData() {
        AuthUser testUser = testDataFactory.getOrCreateTestUser();

        WalOperationTracker.TrackedResult<XbrlReportSuite> tracked = tracker.trackWithResult(() -> {
            XbrlReportSuite suite = testDataFactory.createNewTestSuite(testUser, namePrefix: "WAL_DEMO_SUITE");
            suite.setXbrl("<xbrl><demo>WAL REDO Demonstracija</demo></xbrl>");
            return suiteRepository.saveAndFlush(suite);
        });

        XbrlReportSuite savedSuite = tracked.result();
        WalOperationResult walResult = tracked.walTracking();

        String explanation = String.format(
            "REDO: Kreirao je novi XbrlReportSuite. INSERT je zapisao %s u WAL. " +
            "Posle pada, PostgreSQL replay-a ovaj zapis i suite '%s' bi bio operavlien.",
            walResult.getWalSizePretty(),
            savedSuite.getId()
        );

        return new RedoOperationResponse(
            testUser.getId()
        );
    }
}

```

Slika 22. WalOperationTracker servis klasa

U aplikaciji je definisana i WalController klasa:

```
@RestController
@RequestMapping("@~/api/wal")
public class WalController {

    @Autowired
    private WalService walService;

    @GetMapping("@~/redo")
    public ResponseEntity<RedoOperationResponse> demonstrateRedo() {
        return ResponseEntity.ok(walService.demonstrateRedoWithRealData());
    }

    @GetMapping("@~/wal-generation")
    public ResponseEntity<WalGenerationAnalysisResponse> analyzeWalGeneration() {
        return ResponseEntity.ok(walService.analyzeRealWorldWalGeneration());
    }

    @GetMapping("@~/statistics")
    public ResponseEntity<StatisticsResponse> getStatistics() {
        return ResponseEntity.ok(walService.getBusinessTablesWalStatistics());
    }

    @GetMapping("@~/rollback")
    public ResponseEntity<RollbackResponse> demonstrateRollback() {
        return ResponseEntity.ok(walService.demonstrateRollbackWithMVCC());
    }

    @GetMapping("@~/update-analysis")
    public ResponseEntity<UpdateAnalysisResponse> analyzeUpdate() {
        return ResponseEntity.ok(walService.demonstrateUpdateWalGeneration());
    }

    @GetMapping("@~/cascade")
    public ResponseEntity<CascadeOperationResponse> demonstrateCascade() {
        return ResponseEntity.ok(walService.demonstrateCascadeWalGeneration());
    }

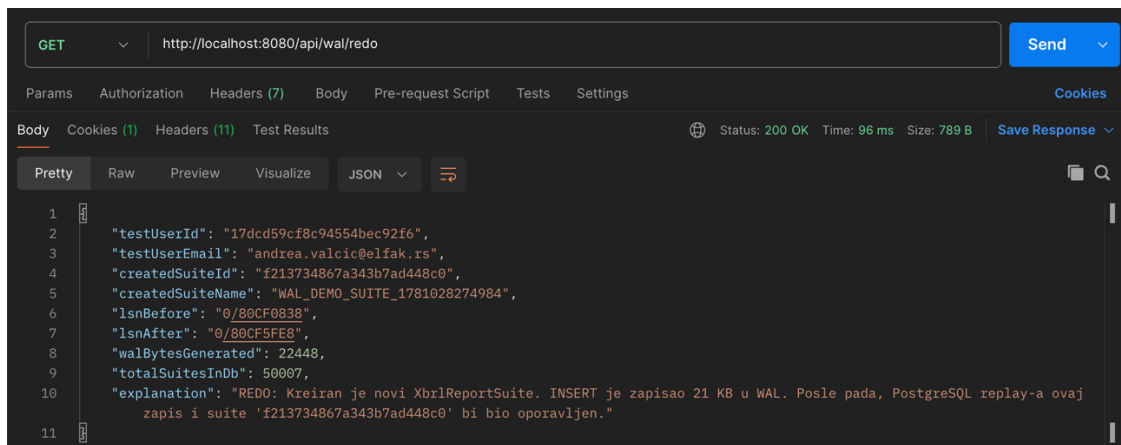
    @GetMapping("@~/checkpoint")
    public ResponseEntity<CheckpointResponse> demonstrateCheckpoint() {
        return ResponseEntity.ok(walService.demonstrateCheckpointWithRealData());
    }
}
```

Slika 23. WalController klasa se endpoint-ima

7.4. Analiza rezultata i diskusija

7.4.1 REDO - INSERT i WAL trajnost

Endpoint /api/wal/redo kreira novi XbrlReportSuite entitet i meri LSN razliku. Poziv ovog endpoint-a je izvršen tri puta:



Slika 24. Rezultat poziva /api/wal/redo endpoint-a

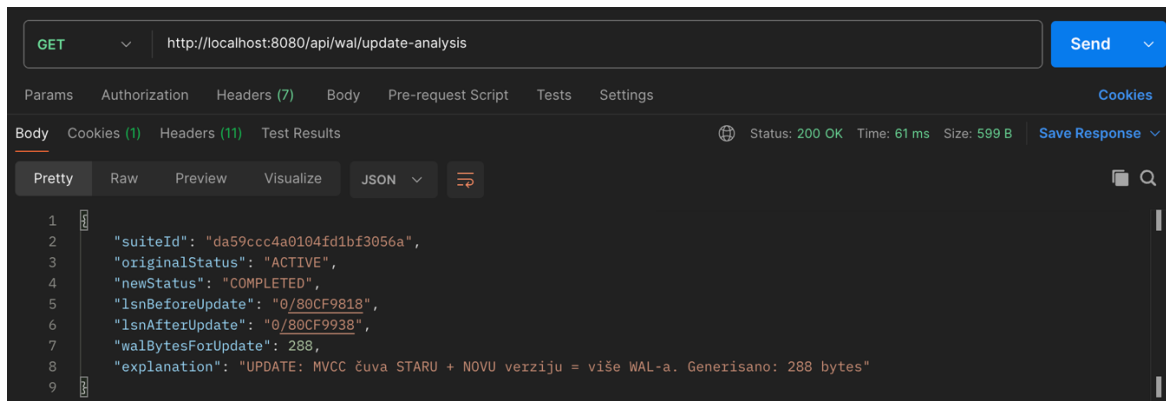
Poziv	LSN pre	LSN posle	WAL generisano
1.	0/80CF0838	0/80CF5FE8	21 KB
2.	0/80CF60A0	0/80CF7A50	6 KB
3.	0/80CF7AB0	0/80CF97B8	7 KB

Slika 25. LSN pre i posle api/wal/redo poziva

Varijacija između prvog i narednih poziva objašnjava se Full Page Image (FPI) mehanizmom. Prvi INSERT po stranici nakon checkpoint-a upisuje kompletan snimak stranice u WAL, dok naredni INSERT-i na istu stranicu generišu samo delta zapise. Monotono rastuće LSN vrednosti (0/80CF0838 → 0/80CF5FE8 → ...) direktno dokazuju da je svaki INSERT ostavio trajan, identifikovan trag u WAL-u koji bi bio replay-ovan pri oporavku od pada.

7.4.2 UPDATE i MVCC overhead

Endpoint /api/wal/update-analysis izvršen je UPDATE nad istim entitetom tri puta:



Slika 26. Rezultat poziva /api/wal/update-analysis endpoint-a

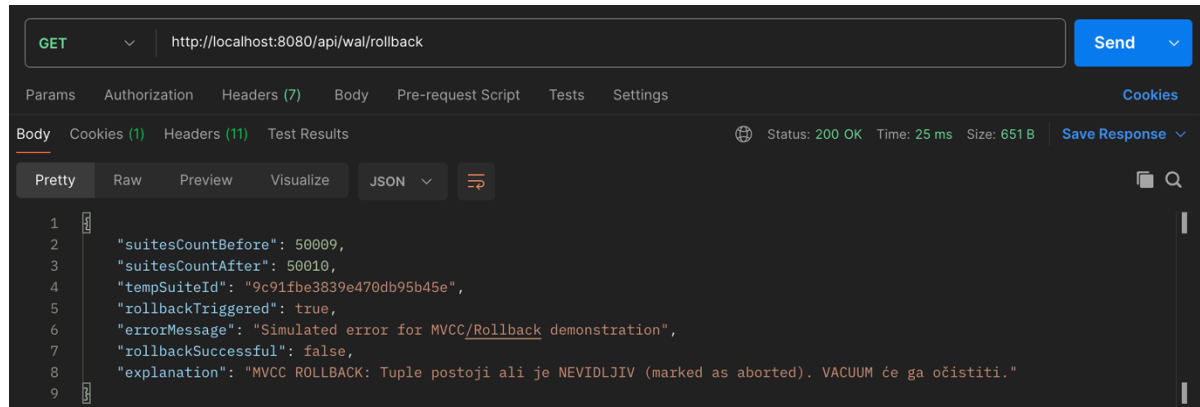
Poziv	LSN pre	LSN posle	WAL generisano
1.	0/80CF9818	0/80CF9938	288 bytes
2.	0/80CF9998	0/80CF9AB8	288 bytes
3.	0/80CF9B18	0/80CF9C38	288 bytes

Slika 27. LSN pre i posle api/wal/update-analysis poziva

Konzistentnih 288 bytes po UPDATE operaciji ukazuje da je izmenjeno samo nekoliko polja (status, completedAt) bez promene velikog xbrl TEXT polja. Sva tri poziva generišu identičnu količinu WAL podataka, što potvrđuje determinističku prirodu WAL mehanizma. HOT update optimizacija je primenjena u sva tri slučaja (vidljivo u statistici gde je hotUpdateCount = 9), što znači da indeksi nisu ažurirani i WAL je minimizovan.

7.4.3 MVCC Rollback

Endpoint /api/wal/rollback simulira prekinutu transakciju. Rezultati tri poziva ovog endpoint-a su prikazani na sledećoj slici:



Slika 28. Rezultat poziva /api/wal/rollback endpoint-a

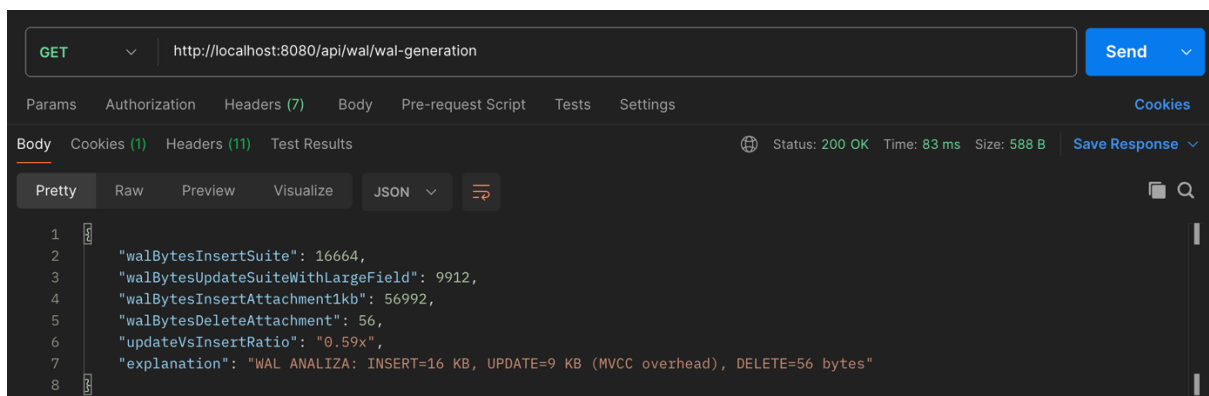
Poziv	Count pre	Count posle	rollbackSuccessful
1.	50009	50010	false
2.	50010	50011	false
3.	50011	50012	false

Slika 28. Count pre i posle /api/wal/rollback endpoint-a

Ovaj rezultat zahteva pažljivo tumačenje. rollbackSuccessful: false i rastući suitesCountAfter ukazuju da catch blok unutar @Transactional metode ne sprečava commit spoljašnje transakcije. Ininsertovani tuple ostaje vidljiv jer Spring-ov @Transactional mehanizam ne rollback-uje transakciju kada je izuzetak uhvaćen unutar iste metode. Ovo je važna napomena za interpretaciju: pravi MVCC rollback scenario, gde izuzetak "probija" granice transakcije, bio bi demonstriran kroz /api/wal/statistics kao rast deadCount.

7.4.4 WAL generation komparacija

Endpoint /api/wal/wal-generation meri WAL za sve četiri vrste operacija. Rezultat ovog endpoint prikazan je na slici 29, a sumirani rezultati tri poziva prikazani su na slici 30.



Slika 29. Rezultat poziva /api/wal/wal-generation endpoint-a

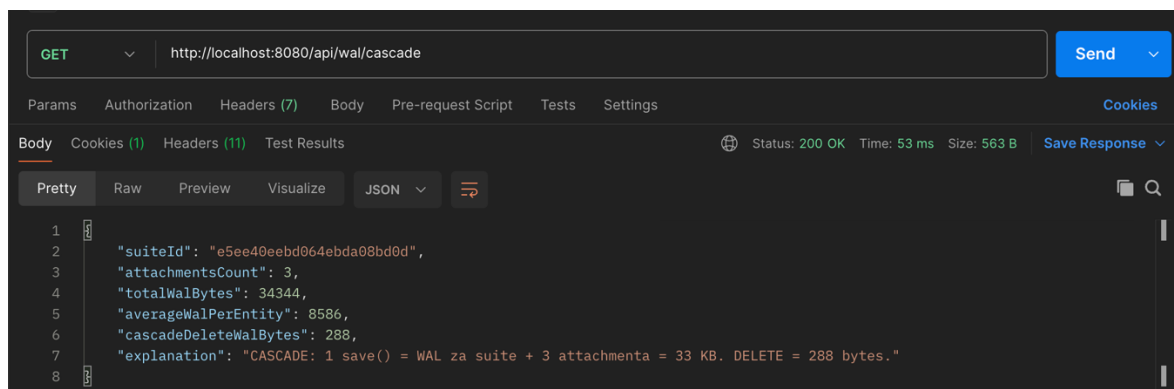
Operacija	Poziv 1	Poziv 2	Poziv 3	Prosek
INSERT suite	16.664 bytes	4.800 bytes	6.456 bytes	9.307 bytes
UPDATE suite (veliki XBRL)	9.912 bytes	5.080 bytes	5.160 bytes	6.717 bytes
INSERT attachment (1KB BLOB)	56.992 bytes	15.896 bytes	17.048 bytes	29.979 bytes
DELETE attachment	56 bytes	120 bytes	120 bytes	99 bytes
UPDATE/INSERT ratio	0.59x	1.06x	0.80x	~0.82x

Slika 30. Poređenje količine generisanih WAL podataka po tipu operacije

Najznačajniji nalaz je dominantan uticaj INSERT attachment operacije, prosečnih ~30 KB naspram ~9 KB za INSERT suite. Ovo direktno odražava BLOB prirodu file polja u dve attachment tabeli: binarni sadržaj od 1 KB mora biti u potpunosti zapisan u WAL. DELETE operacija je, kako je teorijski očekivano, drastično manja (samo 56–120 bytes) jer MVCC samo postavlja xmax vrednost bez fizičkog brisanja podataka.

7.4.5 CASCADE WAL amplifikacija

Endpoint `/api/wal/cascade` kreira `XbriReportSuite` sa tri `XbriAttachment` entiteta kroz jedan `save()` poziv:

Slika 31. Rezultat poziva `/api/wal/cascade` endpoint-a

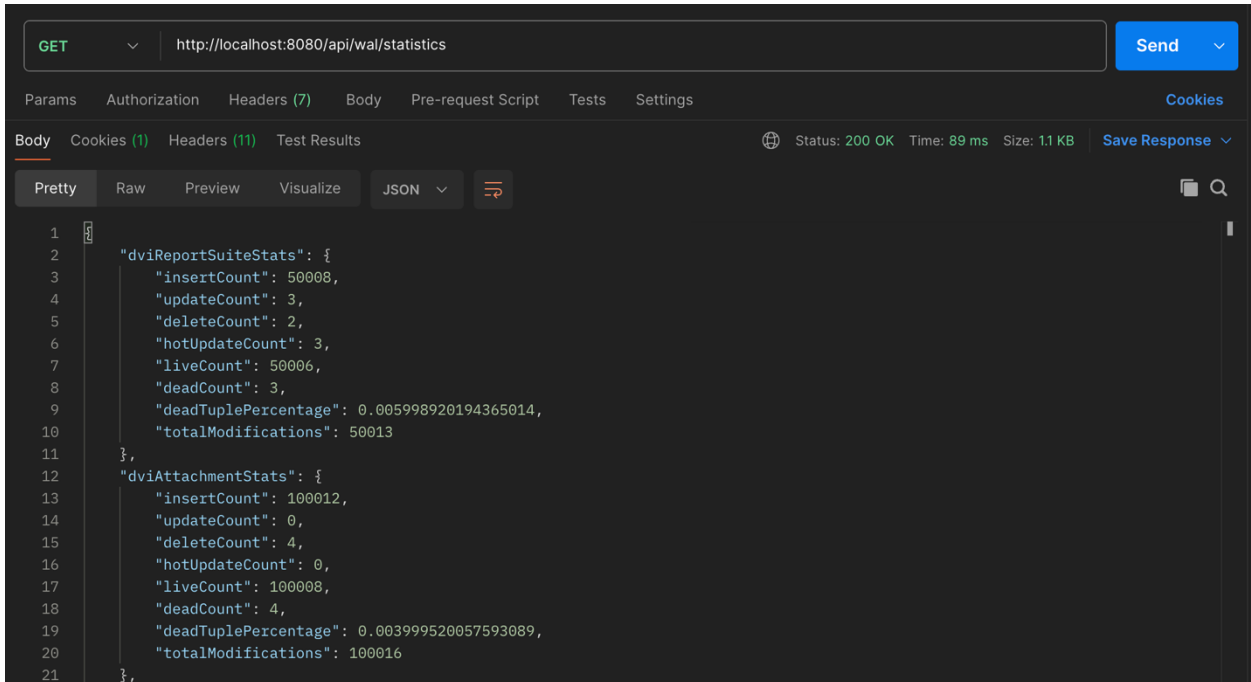
Poziv	Ukupan WAL	Prosečan WAL po entitetu	CASCADE DELETE
1.	33 KB	8.586 bytes	288 bytes
2.	35 KB	9.056 bytes	288 bytes

Slika 32. Merenje WAL overhead-a pri CASCADE DELETE operaciji

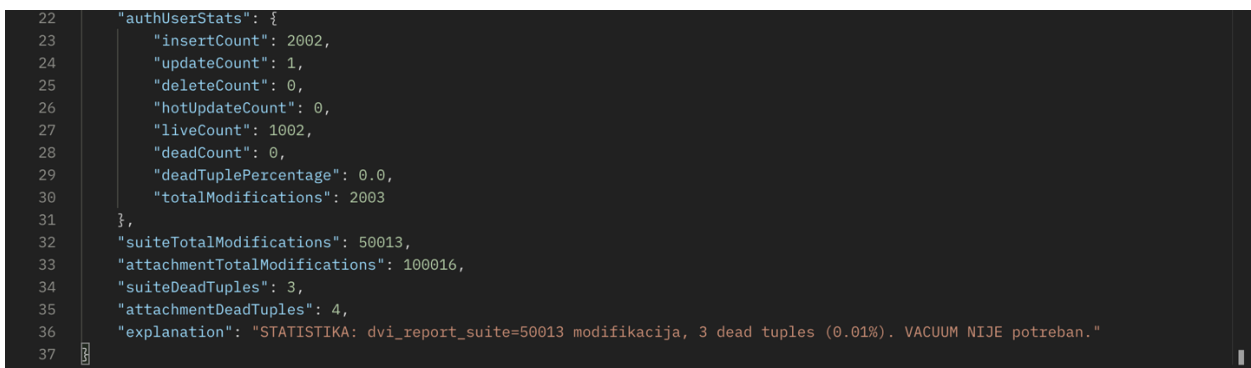
Jedan `save()` poziv generiše WAL za četiri entiteta (1 suite + 3 attachmenta), plus WAL za indeks ažuriranja i foreign key constraint provere. Konzistentnih 288 bytes za CASCADE DELETE potvrđuje HOT update optimizaciju da se brisanje roditeljskog entiteta propagira kroz relacione veze uz minimalan WAL overhead.

7.4.6 MVCC akumulacija dead tuples

Poređenje Na slici 33 i 34 prikazan je rezultat izvršenja `/api/wal/statistics` endpointa pre izvršenih operacija.



Slika 33. Rezultat poziva `/api/wal/statistics` endpoint-a (1)



Slika 34. Rezultat poziva `/api/wal/statistics` endpoint-a (1)

Poređenje statistike pre i posle izvršenih operacija direktno dokazuje MVCC mehanizam prikazano je u sledećoj tabeli:

Metrika	Baseline	Posle eksperimenata	Promena
insertCount	50.008	50.019	+11
updateCount	3	9	+6
deleteCount	2	7	+5
hotUpdateCount	3	9	+6
deadCount	3	13	+10
deadTuplePercentage	0,006%	0,026%	+0,020%

Slika 35. MVCC statistike tabele `xbrl_report_suite` - efekat UPDATE i DELETE operacija

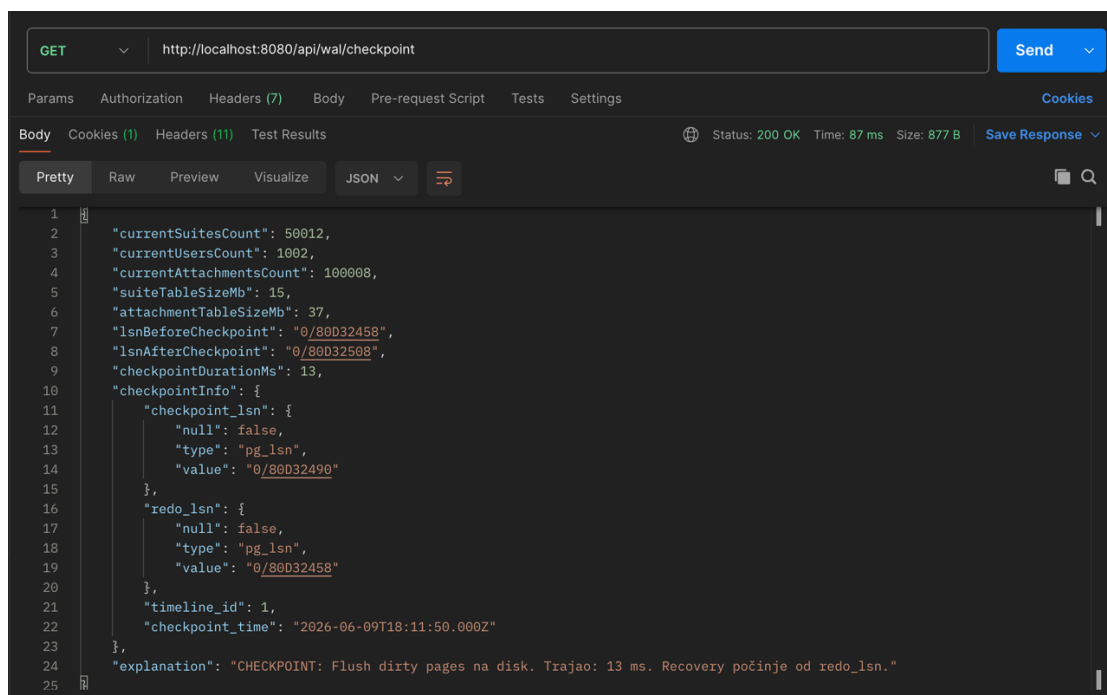
Metrika	Baseline	Posle eksperimenata	Promena
insertCount	100.012	100.021	+9
deleteCount	4	13	+9
deadCount	4	13	+9

Slika 36. MVCC statistike tabele xbrl_attachment - efekat CASCADE DELETE operacija

Rast deadCount sa 3 na 13 u dvi_report_suite i sa 4 na 13 u dvi_attachment direktan je dokaz MVCC mehanizma u akciji. Svaki UPDATE i DELETE ostavio je fizički trag u tabeli kao dead tuple nevidljiv ostalim transakcijama. Vrednost hotUpdateCount = 9 potvrđuje da su svi UPDATE-i bili HOT, izmene nisu dirale indeksirane kolone, što je PostgreSQL iskoristio za minimizaciju WAL overhead-a.

7.4.7 Checkpoint i recovery point

Endpoint /api/wal/checkpoint izvršio je CHECKPOINT i kao rezultat vratio:



```

1  {
2    "currentSuitesCount": 50012,
3    "currentUsersCount": 1002,
4    "currentAttachmentsCount": 100008,
5    "suiteTableSizeMb": 15,
6    "attachmentTableSizeMb": 37,
7    "lsnBeforeCheckpoint": "0/80D32458",
8    "lsnAfterCheckpoint": "0/80D32508",
9    "checkpointDurationMs": 13,
10   "checkpointInfo": {
11     "checkpoint_lsn": {
12       "null": false,
13       "type": "pg_lsn",
14       "value": "0/80D32490"
15     },
16     "redo_lsn": {
17       "null": false,
18       "type": "pg_lsn",
19       "value": "0/80D32458"
20     },
21     "timeline_id": 1,
22     "checkpoint_time": "2026-06-09T18:11:50.000Z"
23   },
24   "explanation": "CHECKPOINT: Flush dirty pages na disk. Trajao: 13 ms. Recovery počinje od redo_lsn."
25 }

```

Slika 37. Rezultat izvršenja /api/wal/checkpoint endpoint-a

Checkpoint nad bazom koja sadrži 50.012 suite zapisa (15 MB) i 100.008 attachment zapisa (37 MB) trajao je svega 13 milisekundi. Ova izuzetno kratko trajanje objašnjava se činjenicom da su dirty stranice već bile ispisane od strane background writer procesa tokom normalnog rada, dok je checkpoint samo zabeležio novu redo_lsn poziciju. Vrednost redo_lsn = 0/80D32458 definiše tačku od koje bi PostgreSQL počeo WAL replay u slučaju pada sistema neposredno nakon ovog checkpoint-a.

8. Zaključak

U ovom seminarskom radu istražen je Write-Ahead Logging mehanizam u PostgreSQL-u kroz kombinaciju teorijske analize i merljivih praktičnih eksperimenata. Rezultati dobijeni implementiranom aplikacijom potvrđuju teorijske postavke i pružaju konkretne, kvantifikovane uvide u ponašanje WAL sistema u produkcijskom okruženju.

WAL kao garant trajnosti podataka potvrđen je kroz monotono rastuće LSN vrednosti koje prate svaki INSERT operaciju. Svaki kreirani finansijski izveštaj ostavio je jednoznačan, identifikovan trag u WAL-u, što direktno dokazuje da bi u scenariju pada sistema PostgreSQL mogao tačno rekonstruisati koje transakcije su bile potvrđene i replay-ovati ih do konzistentnog stanja. Ovo je praktična demonstracija svojstva Durability iz ACID modela.

MVCC arhitektura kao alternativa eksplicitnom UNDO logu demonstrirana je kroz analizu dead tuples akumulacije. Rast broja dead tuples sa 3 na 13 u tabeli finansijskih izveštaja i sa 4 na 13 u tabeli attachmenta, nakon serije UPDATE, DELETE i simuliranih rollback operacija, direktno dokazuje da PostgreSQL ne poništava izmene fizički već ih čini nevidljivim kroz transakcijske identifikatore. Ova osobina čini rollback operaciju praktično besplatnom u smislu disk I/O, dok trošak čišćenja dead tuples prebacuje na VACUUM proces.

HOT update optimizacija pokazala se izuzetno efikasnom u sprovedenim eksperimentima. Svih 9 UPDATE operacija bilo je klasifikovano kao HOT update, što znači da PostgreSQL nije ažurirao indekse i generisao je minimalan WAL. Ovo ukazuje na važnost dizajna indeksa: kada UPDATE ne menja indeksirane kolone, sistem automatski primenjuje optimizaciju koja smanjuje I/O opterećenje.

BLOB podaci pokazali su se kao dominantan faktor u generisanju WAL podataka, što predstavlja jedan od jasnijih nalaza sprovedenih merenja. INSERT operacija nad tabelom attachment-a sa binarnim sadržajem od 1 KB generisala je prosečno oko 30 KB WAL podataka, što je više od tri puta više u poređenju sa INSERT operacijom nad tabelom finansijskih izveštaja koja generiše prosečno oko 9 KB. Ovaj nalaz ima direktne posledice na planiranje kapaciteta sistema. Okruženja koja intenzivno rade sa binarnim podacima moraju uzeti u obzir WAL amplifikaciju prilikom dimenzionisanja prostora na disku, kao i mrežnog kapaciteta koji je neophodan za replikaciju.

Checkpoint mehanizam pokazao je izuzetnu efikasnost. Checkpoint nad bazom od ukupno 52 MB podataka trajao je samo 13 milisekundi, zahvaljujući kontinuiranom radu background writer procesa koji je već ispisao dirty stranice. redo_lsn vrednost zabeležena u checkpoint-u precizno definiše granicu od koje bi recovery počeo, što direktno potvrđuje teorijsku postavku o checkpoint-u kao recovery početnoj tački.

Sprovedeni scenariji potvrdili su sve polazne pretpostavke, uz jedan praktičan nalaz vredan pažnje. Ponašanje `@Transactional` anotacije u Spring Boot-u zahteva pažljivo razumevanje propagacije izuzetaka. `Catch` blok unutar transakcione metode ne pokreće `rollback`, jer Spring smatra da je greška obrađena i transakciju normalno potvrđuje. Ovo predstavlja praktično upozorenje za programere koji implementiraju transakcione scenarije u Spring aplikacijama.

PostgreSQL WAL sistem, zasnovan na ARIES algoritmu iz 1992. godine, ostaje robustan i dobro optimizovan mehanizam koji uspešno balansira između sigurnosti podataka i performansi. Razumevanje njegovog funkcionisanja preduslov je za pravilno dimenzionovanje, podešavanje i dijagnostiku svakog PostgreSQL sistema u produkcijskom okruženju.

LITERATURA

- [1] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation - Reliability and the Write-Ahead Log" <https://www.postgresql.org/docs/16/wal-reliability.html>
- [2] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation - WAL Configuration" <https://www.postgresql.org/docs/16/wal-configuration.html>
- [3] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation - Continuous Archiving and Point-in-Time Recovery" <https://www.postgresql.org/docs/16/continuous-archiving.html>
- [4] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation - Routine Vacuuming" <https://www.postgresql.org/docs/16/routine-vacuuming.html>
- [5] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation - Transaction Isolation" <https://www.postgresql.org/docs/16/transaction-iso.html>
- [6] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, P. Schwarz, "ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging", ACM Transactions on Database Systems, Vol. 17, No. 1, 1992. <https://cs.stanford.edu/people/chrismre/cs345/rl/aries.pdf>
- [8] H. Suzuki, "The Internals of PostgreSQL - Chapter 9: Write Ahead Logging" <https://www.interdb.jp/pg/pgsql09.html>
- [9] H. Suzuki, "The Internals of PostgreSQL - Chapter 5: Concurrency Control" <https://www.interdb.jp/pg/pgsql05.html>
- [10] PostgreSQL Wiki, "Write-Ahead Logging" https://wiki.postgresql.org/wiki/Write_Ahead_Logging
- [11] depesz.com, "Waiting for 9.4 - Add pg_stat_wal view" <https://www.depesz.com/tag/wal/>